

Final Project Report

Memory Maze - Human Vs Ai Pathfinding Game



Project Supervisor

Safid Ullah Shah

Submitted by

F25PROJECTB9784

Muhammad Zaid - Bc220407371

Software Projects & Research Section,
Department of Computer Sciences,
Virtual University of Pakistan

Exordium

In the name of Allah, the Compassionate, the Merciful.

Praise be to Allah, Lord of Creation,
The Compassionate, the Merciful,
King of Judgment-day!

You alone we worship, and to You alone we pray for help,
Guide us to the straight path

The path of those who You have favored,

Not of those who have incurred Your wrath,
Nor of those who have gone astray.

Dedication

I dedicate this project to my parents and family for their constant support, encouragement, and sacrifices throughout my academic journey. Their belief in me gave me the strength and motivation to complete this work successfully.

I also dedicate this project to my supervisor who guided and helped me during difficult stages of this project.

Acknowledgement

First of all, I am thankful to Allah Almighty for giving me the strength, patience, and ability to complete this project successfully.

I would like to express my sincere gratitude to my project supervisor, Safid Ullah Shah, for their valuable guidance and constructive feedback throughout the development of this project. Their experience and suggestions greatly helped in improving the quality of this work.

I am also thankful to the faculty members of the Department of Computer Science at Virtual University of Pakistan for providing the knowledge and resources necessary for this project.

Finally, I would like to thank my family and friends for their encouragement, motivation, and support during the completion of this project.

Preface

This project was developed as part of the requirements for the completion of the Bachelor of Science in Computer Science degree. The purpose of this project is to explore practical problem-solving through software development while applying the concepts learned during academic studies.

The project provided an opportunity to gain hands-on experience in system design, programming, debugging, and project management. Throughout the development process, several challenges were encountered, which helped improve technical knowledge and problem-solving abilities.

This report presents the analysis, design, implementation, and evaluation of the developed system. It is hoped that the project will serve as a useful contribution and provide valuable learning for future improvements and research.

Table of Contents

Chapter No. 1

Gathering & Analyzing Info

1. Introduction
2. Purpose
3. Scope
4. Definitions, acronyms and abbreviations
5. Project requirements
 1. Functional Requirements
 2. Non Function Requirements
6. Use Cases and Usage Scenarios
 1. Use Case Diagrams
 2. Usage Scenarios
7. Development Methodology
 1. Chosen Methodology
 2. Reasons for Chosen Methodology
 3. Work Plan (Gannt Chart)
 4. Project Schedule (Submission Calendar)

Chapter No. 2

Designing The Project

1. Introduction
2. Purpose
3. Scope
4. Definitions, acronyms and abbreviations
5. Architectural Representation (Architecture Diagram)
6. Dynamic Model: Sequence Diagrams

7. Object Model/Logical Model: Class Diagram
8. Database Model (Database Diagram)
9. Graphical User Interfaces

Chapter No. 3

Development

1. Development plan (Architecture Diagram)

Chapter 1

Introduction

This document provides the foundation for the *Memory Maze - Human vs AI Pathfinding Game* project.

Purpose

This document specifies the requirements for the *Memory Maze - Human vs AI Pathfinding Game*, where a human player competes against an AI rival to solve dynamically generated mazes. It is intended for developers and supervisors involved in designing, implementing, and testing the system.

Scope

Memory Maze - Human vs AI Pathfinding Game aims to develop a competitive maze-solving game where a human player competes against an AI rival. At the beginning, the maze is displayed fully for a few seconds, after which parts of it disappear. The player must rely on memory and strategy to find the exit. Simultaneously, the AI rival uses the A* Search Algorithm (Manhattan Distance heuristic) to navigate the maze efficiently.

The game tests human cognitive skills against algorithmic precision, providing engaging experience and demonstrating pathfinding, algorithm design, and AI vs human performance comparison while keeping track of results. Additional complexity is introduced by multiple maze sizes, increasing difficulty levels, and power-ups, which can hinder or help in progress.

Definitions, Acronyms and Abbreviations

- UC: User Characteristic (e.g. UC1 means user characteristic 1)
- FR: Functional Requirement (e.g. FR1 means functional requirement 1)
- NFR: Non Functional Requirement (e.g. NFR1 means non functional requirement 1)
- A*: Path finding algorithm
- Manhattan distance heuristic: A calculation useful in grid based pathfinding algorithms
- Python: Programming language
- PyGame: A library which integrates with Python

Project Requirements

Functional Requirements

- FR1: The system shall generate random mazes using recursive backtracking or Prim's algorithm.
- FR2: The maze shall be fully visible for a defined time (e.g., 5–10 seconds).
- FR3: After the reveal time, walls shall disappear partially, requiring the player to rely on memory.
- FR4: Maze complexity (size and branching factor) shall increase with levels.
- FR5: The player shall move using keyboard arrow keys.
- FR6: The player shall be allowed to collect power-ups (e.g., reveal part of maze, slow down AI).

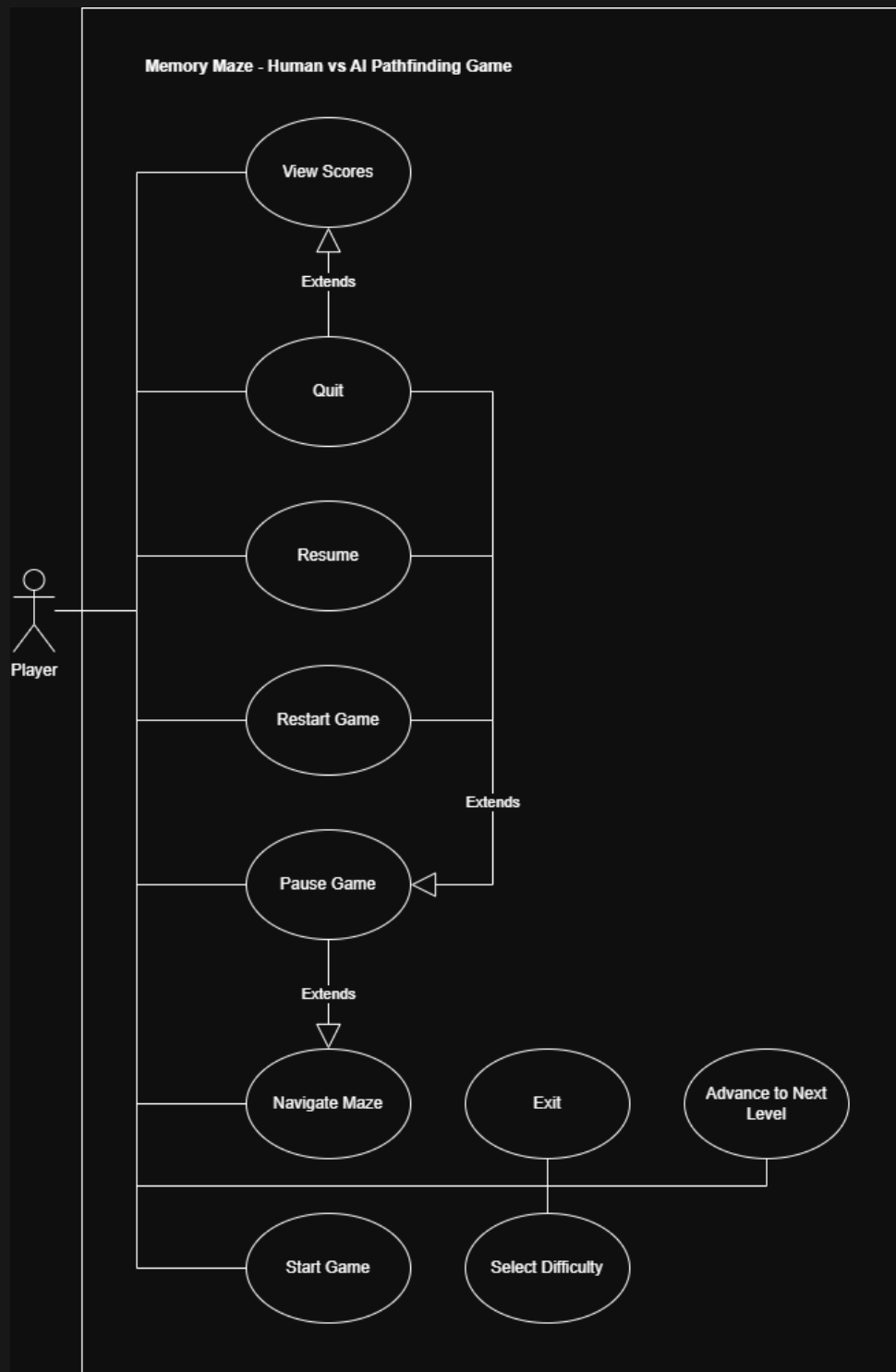
- FR7: The player shall be penalized for hitting dead ends (e.g., time penalty).
- FR8: The AI rival shall use the A* Search Algorithm with Manhattan Distance heuristic to compute its shortest path.
- FR9: The AI shall update its path dynamically if obstacles appear or change.
- FR10: The AI's speed shall increase with higher levels to maintain difficulty.
- FR11: The game shall run both player and AI simultaneously, updating positions in real-time.
- FR12: The game shall declare a winner when either player or AI reaches the exit first.
- FR13: The game shall maintain a scoreboard (wins, losses, fastest completion times).
- FR14: The game shall support multiple difficulty levels (small, medium, large mazes).
- FR15: The system shall allow the player to restart, pause, and exit at any time.

Non Function Requirements

- NFA1: The game shall support 30 frames per second.
- NFA2: The game shall be designed primarily for Windows environments and is not required to support other operating systems.
- NFA3: The game shall maintain stable gameplay without crashes during a continuous session of 30 minutes.
- NFA4: The game interface shall provide clear visual feedback for player actions and game state changes.
- NFA5: The system shall support single-player gameplay only, involving one human player competing against one AI agent.

Use Cases and Usage Scenarios

Use case Diagrams



Usage Scenarios

Select Difficulty

Use Case Name: Select Difficulty

Use Case ID: UseCase1

Actor: Player

Summary: This use case describes the scenario in which player selects the game difficulty.

Pre-Condition: Player must be on the main menu.

Post-Condition: Player is back on the main menu.

Extend: N/A

Uses: N/A

Normal Course of Events: Player will trigger the Select Difficulty button, then trigger one of the difficulty buttons (easy, medium, hard) and will be taken back to main menu.

Alternative Path: N/A

Exception: N/A

Start Game

Use Case Name: Start Game

Use Case ID: UseCase2

Actor: Player

Summary: This use case describes the scenario in which player starts playing the game.

Pre-Condition: Player must be on main menu.

Post-Condition: Player can see the maze on playing screen.

Extend: N/A

Uses: N/A

Normal Course of Events: Player triggers the Play button.

Alternative Path: Player can trigger Select Difficulty button to select difficulty, come back on the main menu and then trigger Play button.

Exception: N/A

Navigate Maze

Use Case Name: Navigate Maze

Use Case ID: UseCase3

Actor: Player

Summary: This use case describes the scenario in which player uses arrow keys to navigate through the maze.

Pre-Condition: Maze must be visible.

Post-Condition: Player can move the avatar around.

Extend: N/A

Uses: N/A

Normal Course of Events: Player uses arrow keys to control movements.

Alternative Path: N/A

Exception: N/A

Pause Game

Use Case Name: Pause Game

Use Case ID: UseCase4

Actor: Player

Summary: This use case describes the scenario in which player pauses the game.

Pre-Condition: Maze must be visible.

Post-Condition: Player is taken to pause menu.

Extend: Navigate Maze

Uses: N/A

Normal Course of Events: Player presses the Escape key and is taken to the pause menu.

Alternative Path: N/A

Exception: N/A

Resume

Use Case Name: Resume

Use Case ID: UseCase5

Actor: Player

Summary: This use case describes the scenario in which player can resume the paused game.

Pre-Condition: Player must be on pause menu.

Post-Condition: Maze is visible again.

Extend: Pause Game

Uses: N/A

Normal Course of Events: Player triggers the Resume button and is taken back into the playing state.

Alternative Path: Player presses the Escape key and is taken back into the playing state.

Exception: N/A

Restart Game

Use Case Name: Restart Game

Use Case ID: UseCase6

Actor: Player

Summary: This use case describes the scenario in which player resets the whole level.

Pre-Condition: Player must be on pause menu.

Post-Condition: Maze is visible again but level is reset.

Extend: Pause Game

Uses: N/A

Normal Course of Events: Player triggers the Restart button. Level is reset and game is resumed.

Alternative Path: N/A

Exception: N/A

View Scores

Use Case Name: View Scores

Use Case ID: UseCase7

Actor: Player

Summary: This use case describes the scenario in which player can view their scores.

Pre-Condition: Player must be on main menu or result menu.

Post-Condition: Scoreboard is visible.

Extend: N/A

Uses: N/A

Normal Course of Events: Player triggers the Scoreboard button on the result screen and is taken to the scoreboard menu.

Alternative Path: Player triggers the Scoreboard button on the main menu and is taken to the scoreboard menu.

Exception: N/A

Advance to next Level

Use Case Name: Advance to Next Level

Use Case ID: UseCase8

Actor: Player

Summary: This use case describes the scenario in which player advances the levels.

Pre-Condition: Player must be on result menu.

Post-Condition: Next level is visible.

Extend: N/A

Uses: N/A

Normal Course of Events: On the result menu, player triggers the Next Level button and is taken to the next level.

Alternative Path: N/A

Exception: N/A

Quit

Use Case Name: Quit

Use Case ID: UseCase9

Actor: Player

Summary: This use case describes the scenario in which player is taken to the main menu.

Pre-Condition: Player must be on any of the menus.

Post-Condition: Main menu is visible.

Extend: Pause, View Scores

Uses: N/A

Normal Course of Events: From any menu, player triggers the Quit button and is taken to the main menu.

Alternative Path: N/A

Exception: N/A

Exit

Use Case Name: Exit

Use Case ID: UseCase10

Actor: Player

Summary: This use case describes the scenario in which player terminates the game.

Pre-Condition: Player must be on the main menu.

Post-Condition: Game terminates.

Extend: N/A

Uses: N/A

Normal Course of Events: Player triggers the Exit button and game is terminated.

Alternative Path: N/A

Exception: N/A

Development Methodology

Software development process often refers to the high-level process that governs the development of a software system from its beginning to its end of life – known as a methodology, model or framework.

Various methodologies have been devised, including waterfall, spiral, agile, rapid prototyping, incremental and synchronize and stabilize.

Chosen Methodology

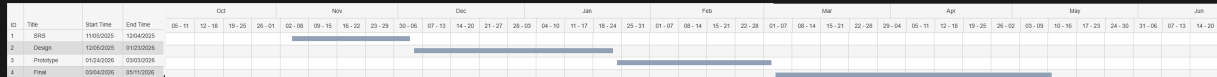
The iterative methodology was chosen for the Memory Maze - Human vs AI Pathfinding Game.

The basic idea behind this method is to develop a system through repeated cycles (iterative) and in smaller portions at a time (incremental), allowing software developers to take advantage of what was learned during development of earlier parts or versions of the system. Learning comes from both the development and use of the system, where possible key steps in the process start with a simple implementation of a subset of the software requirements and iteratively enhance the evolving versions until the full system is implemented

Reasons for Chosen Methodology

- Features can be developed independently and integrated gradually.
- The approach allows continuous testing and debugging.
- It provides flexibility for improving gameplay experience.
- It allows developer to get familiar with the tools.

Work Plan (Ganntt Chart)



Project Schedule (Submission Calendar)

Deliverable	Start Date	End Date	Status
SRS Document	5th November 2025	4th December 2025	Completed
Design Document	5th December 2025	23rd Jan 2026	Completed
Prototype Phase	24th January 2026	3rd March 2026	Completed
Final Deliverable	4th March 2026	11th May 2026	Completed

Chapter 2

Introduction

This Design Document describes the detailed system design of “Memory Maze – Human vs AI Pathfinding Game”, developed as a Final Year Project. It explains how the requirements defined in the Software Requirements Specification (SRS) are converted into a structured and implementable system design.

Purpose

The purpose of the design phase is to establish a clear blueprint for development by identifying system components, defining their responsibilities, and specifying interactions before implementation begins. This phase helps reduce ambiguity, detect design issues early, and improve system maintainability through a modular and organized structure.

By providing visual and structural representations of the system, the design phase enhances communication among developers and evaluators and ensures efficient and reliable implementation of “Memory Maze – Human vs AI Pathfinding Game”.

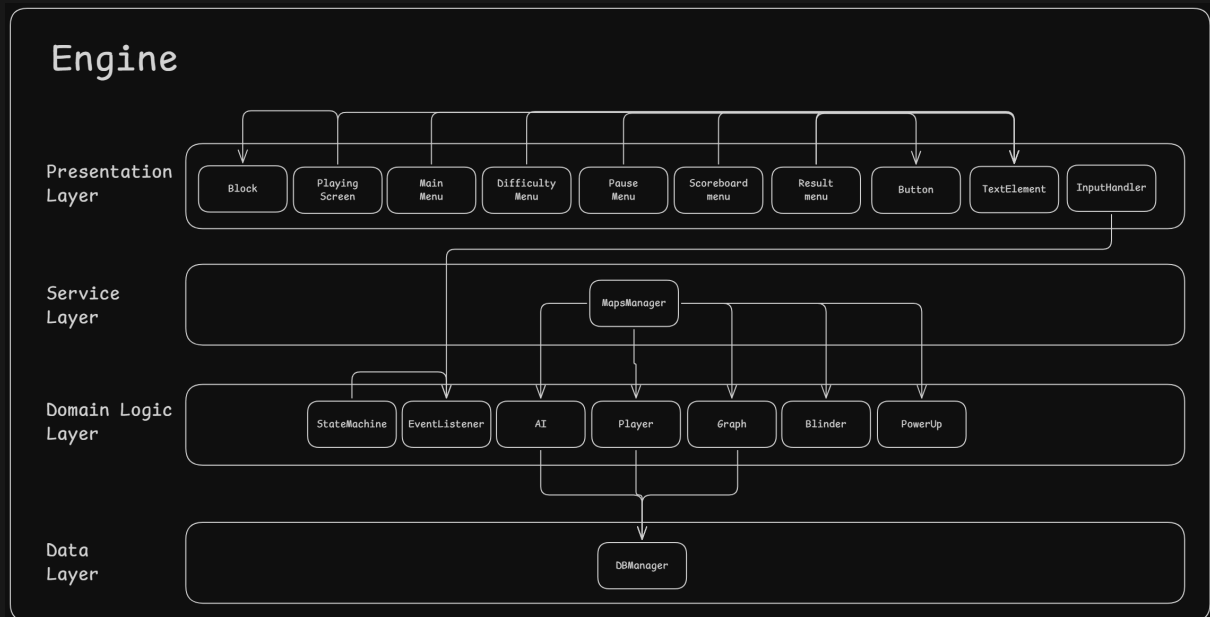
Scope

This document includes the

- Entity Relationship Diagram (ERD) to define the data structure
- Sequence diagrams to illustrate interactions among system components
- An architecture design diagram to present the overall system organization
- Class diagrams to represent the object-oriented structure of the system.
- Database design for data storage
- Interface design to describe user interaction
- Test cases to validate the correctness of the proposed design

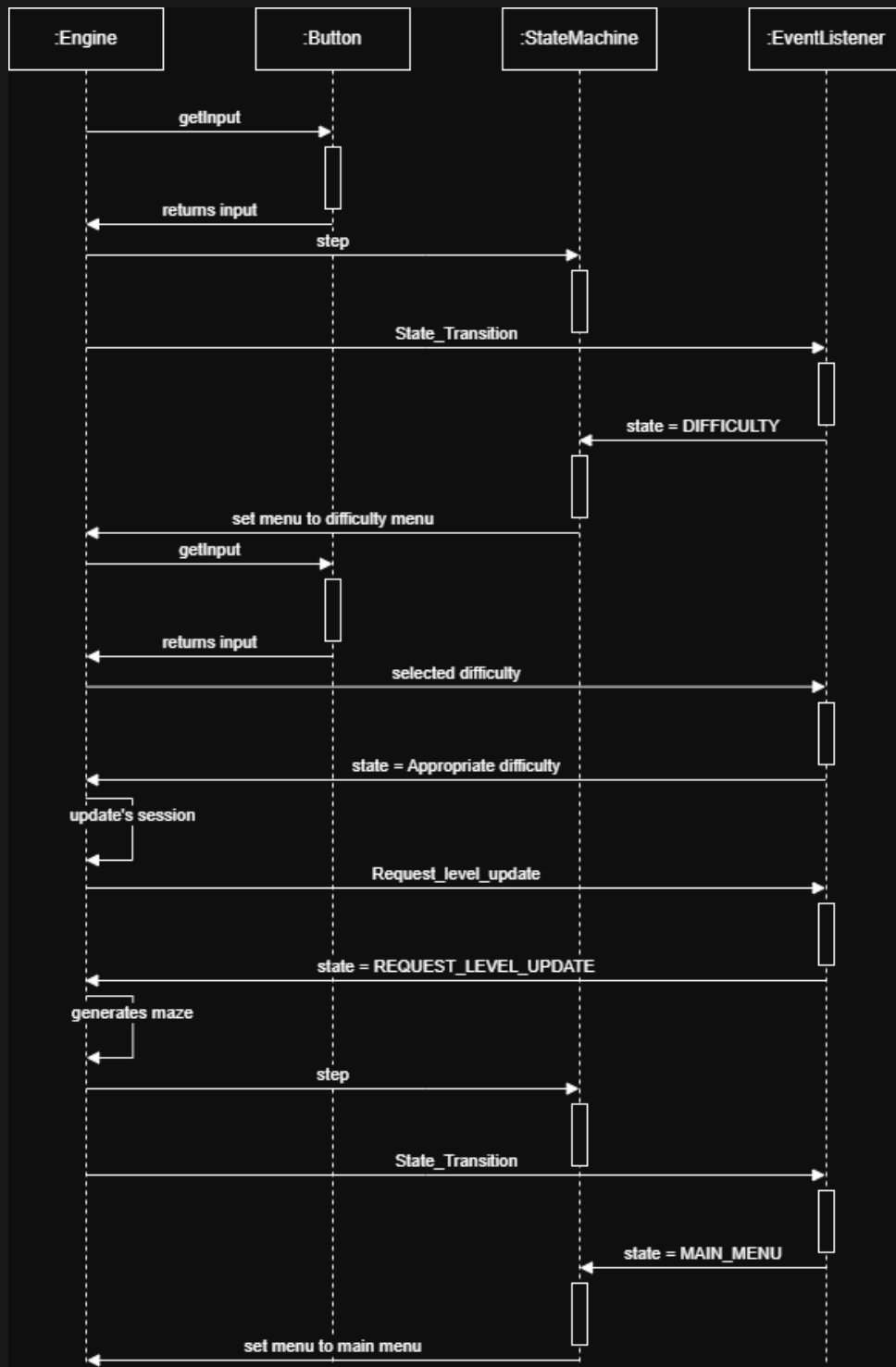
Definitions, Acronyms and Abbreviations

Architectural Representation (Architecture Diagram)

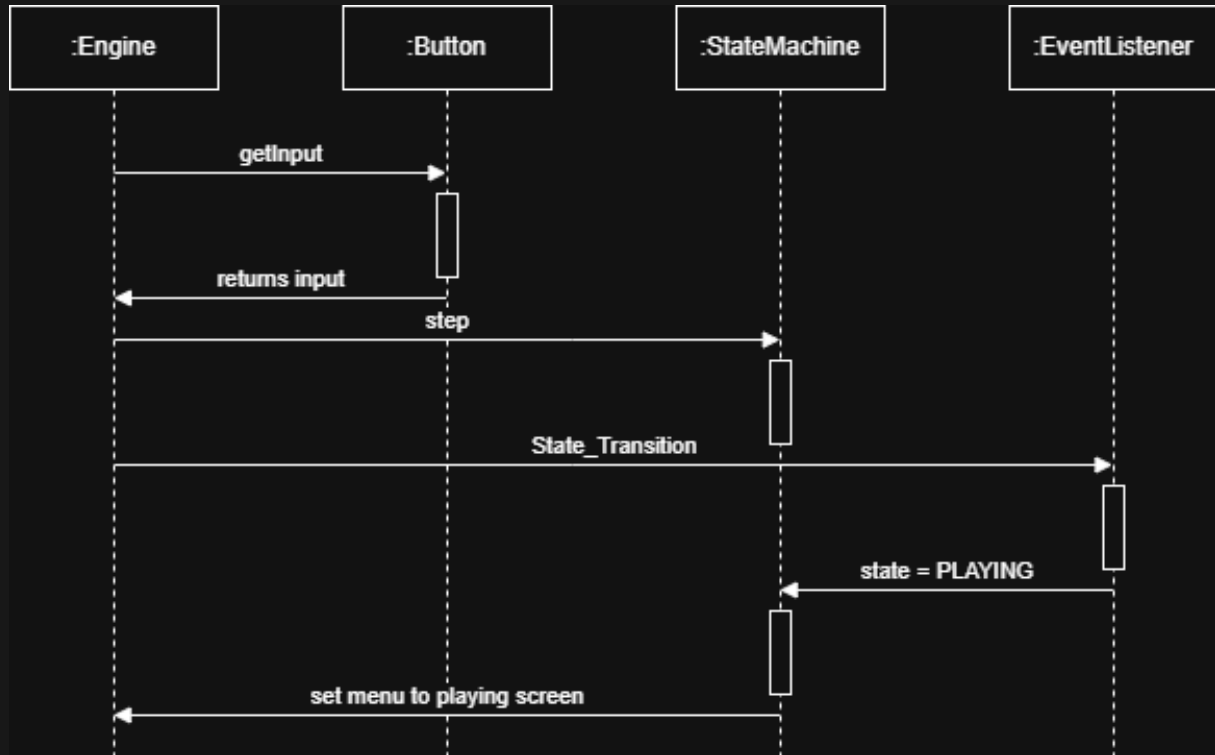


Dynamic Model: Sequence Diagrams

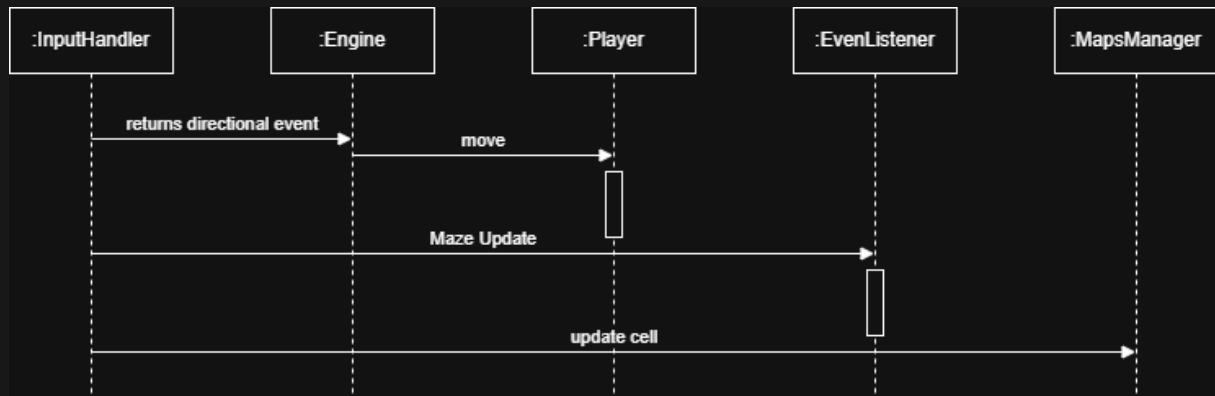
Select Difficulty



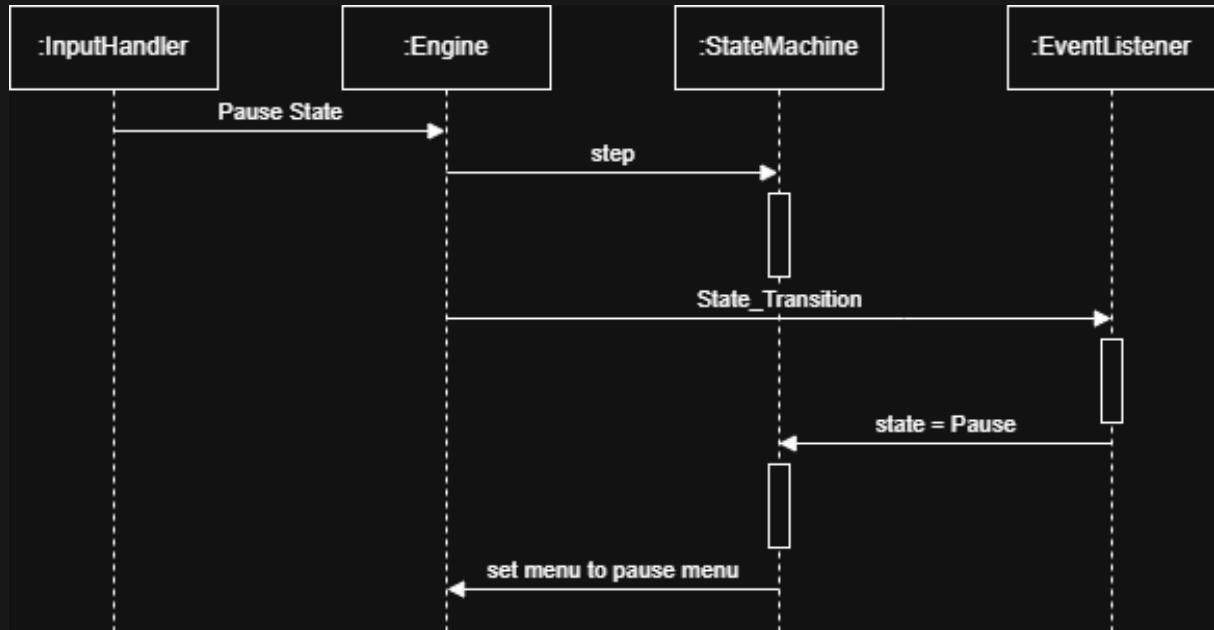
Start Game



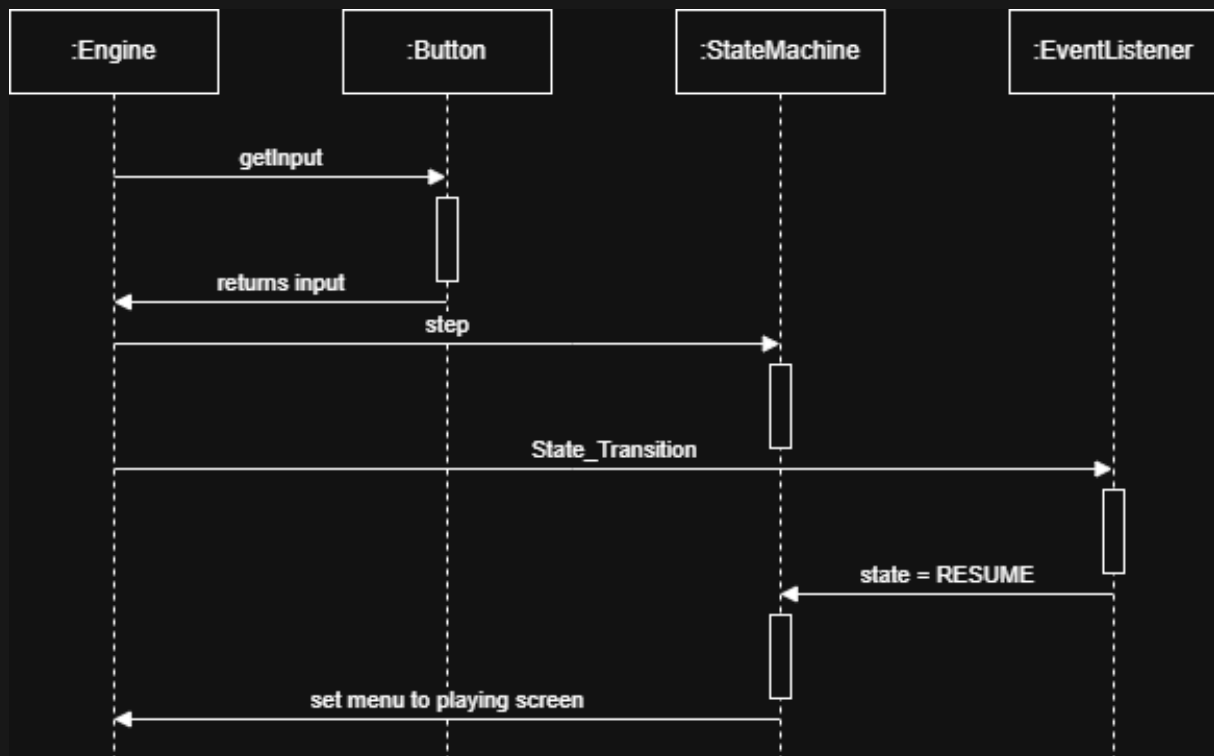
Navigate Maze



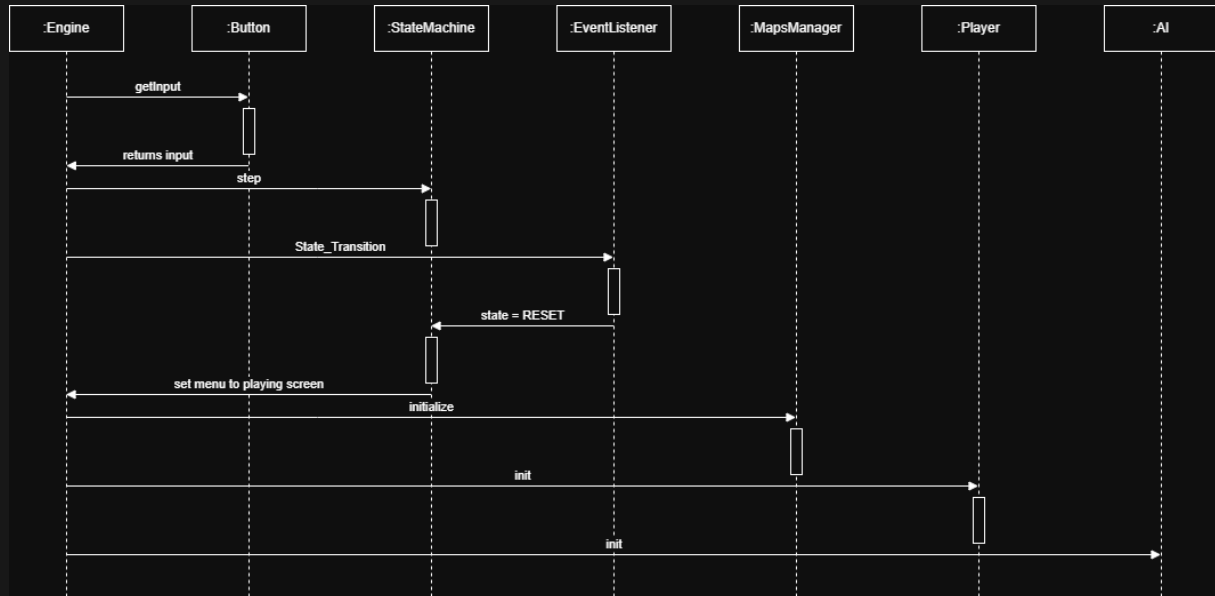
Pause Game



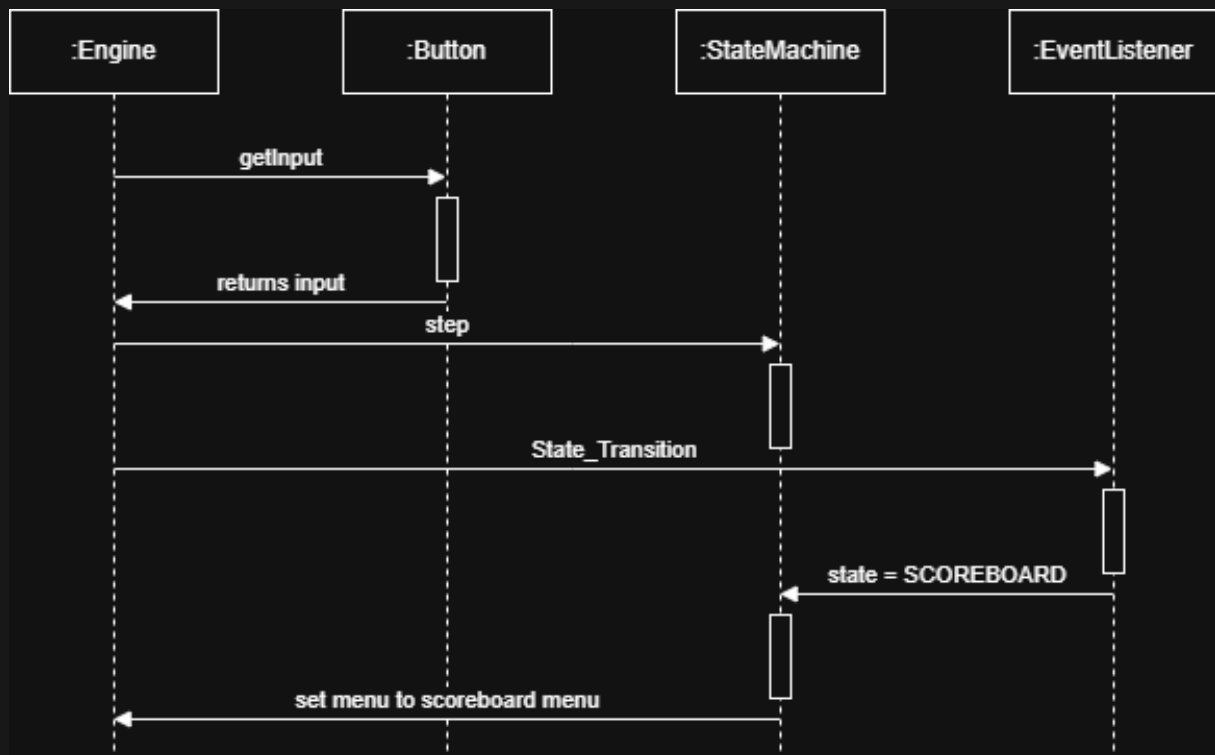
Resume



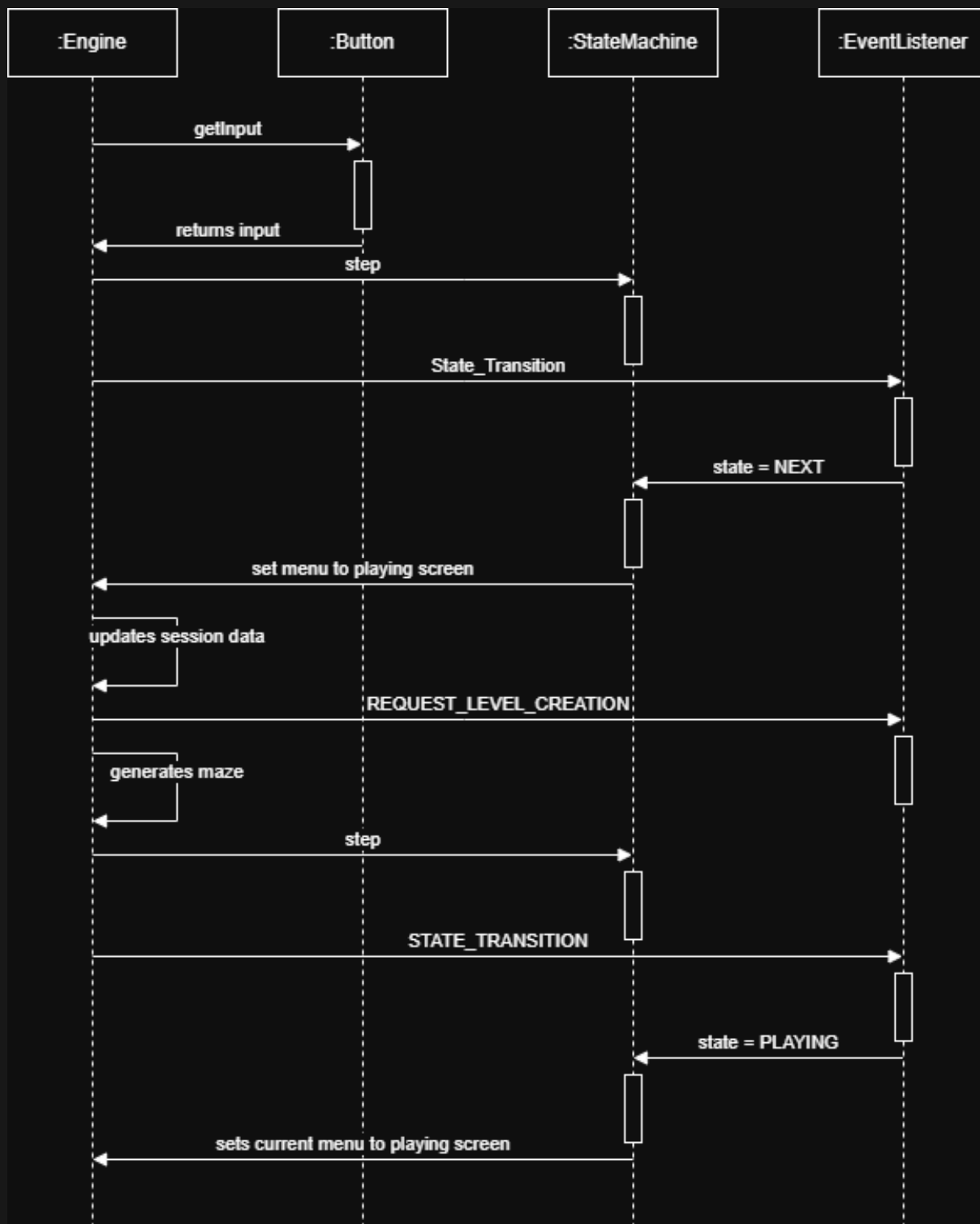
Restart Game



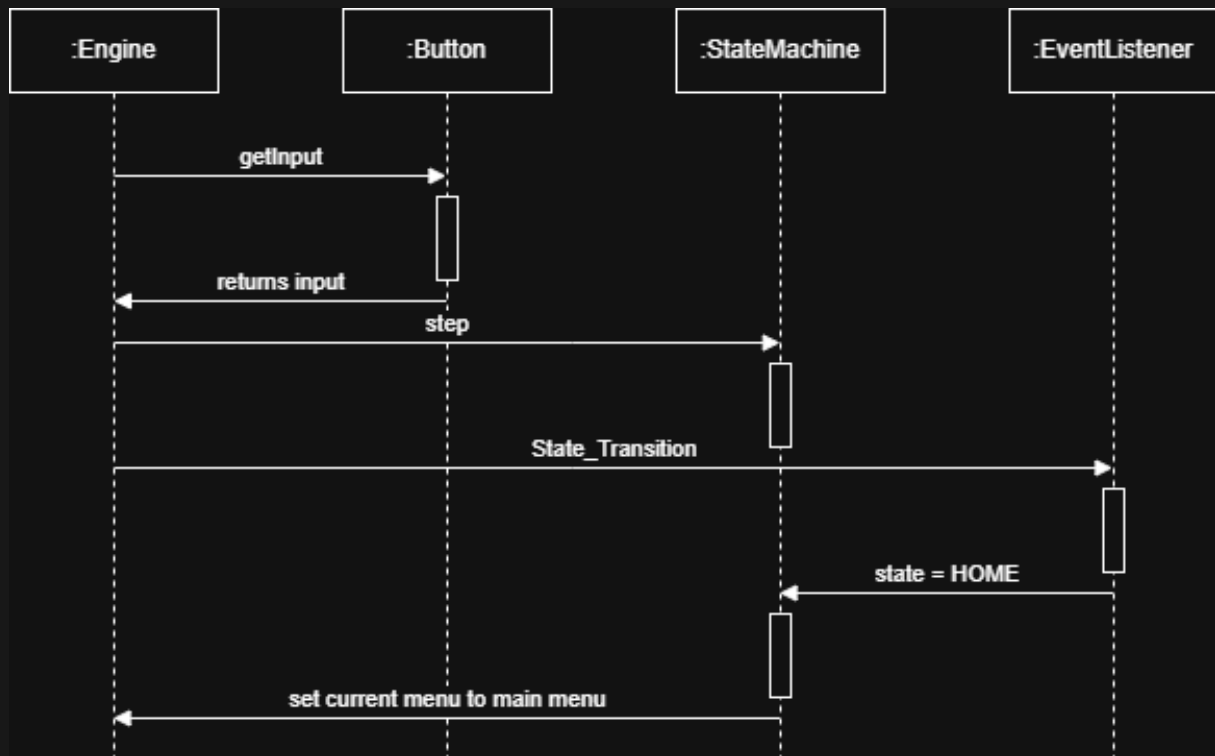
View Scores



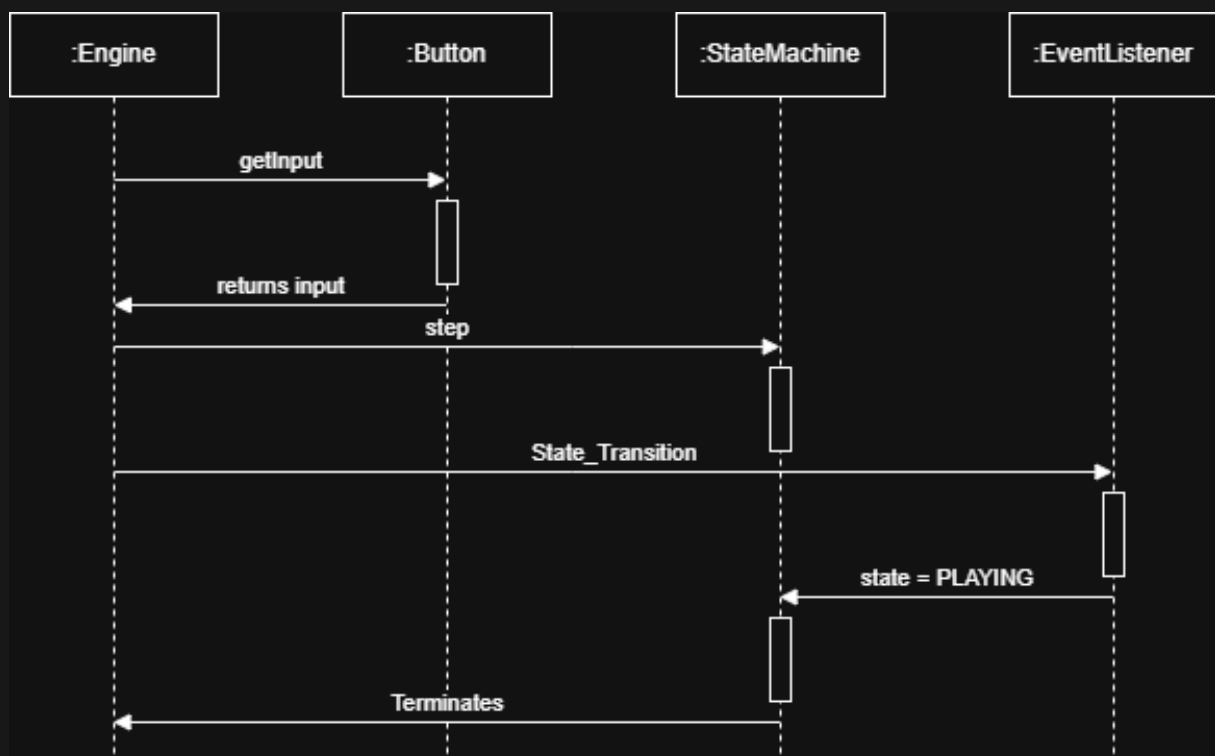
Advance to next Level



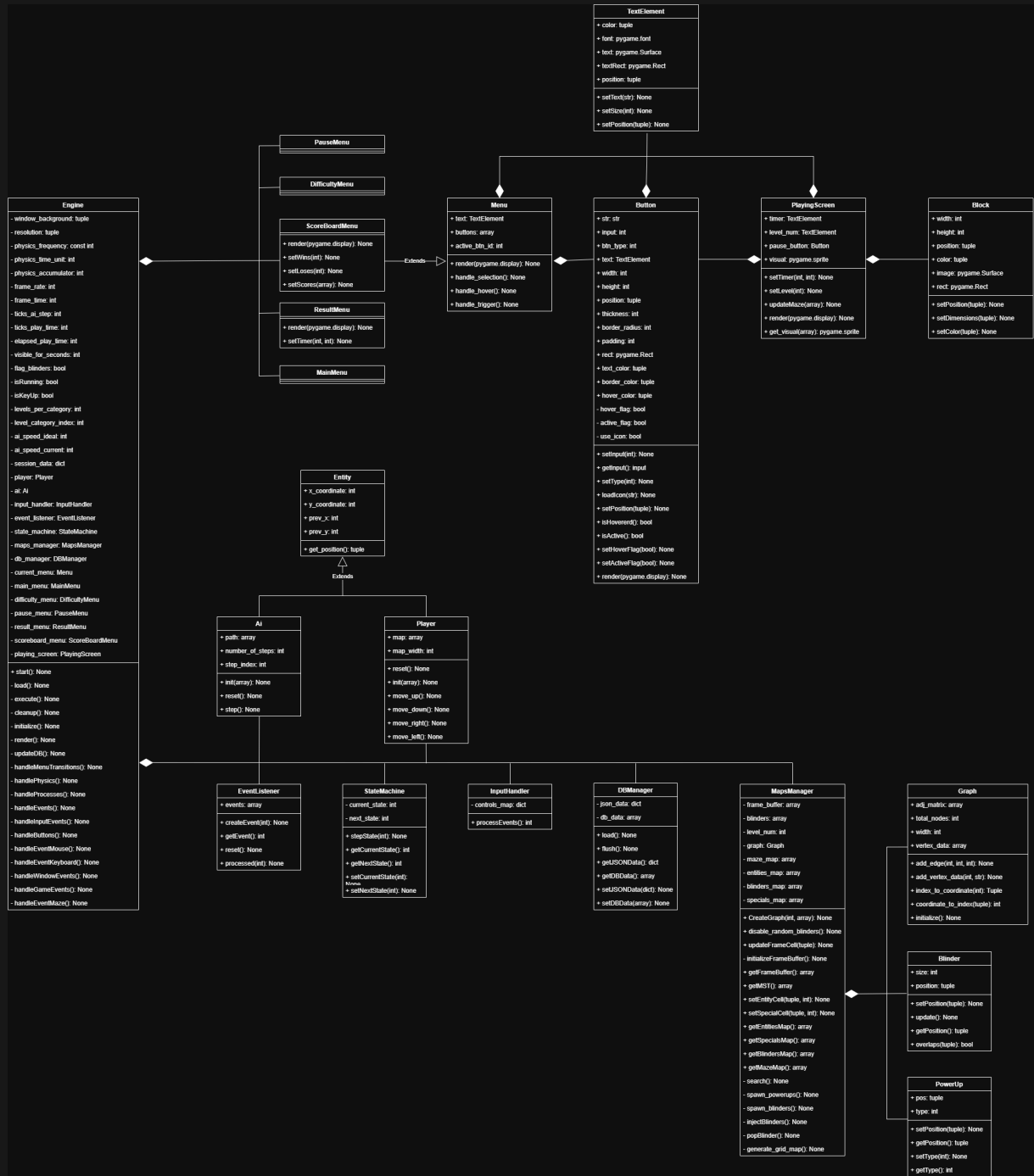
Quit



Exit



Object Model/Logical Model: Class Diagram



Database Model (Database Diagram)



Graphical User Interfaces

Main Menu



Difficulty Menu

Difficulty Menu

Easy

Medium

Hard

Scoreboard Menu



Score Board Menu

99:99:99

99:99:99

99:99:99

99:99:99

99:99:99

Wins: 0

Loses: 0

Playing

Timer: 00:07

Level: 1

||

Pause Menu

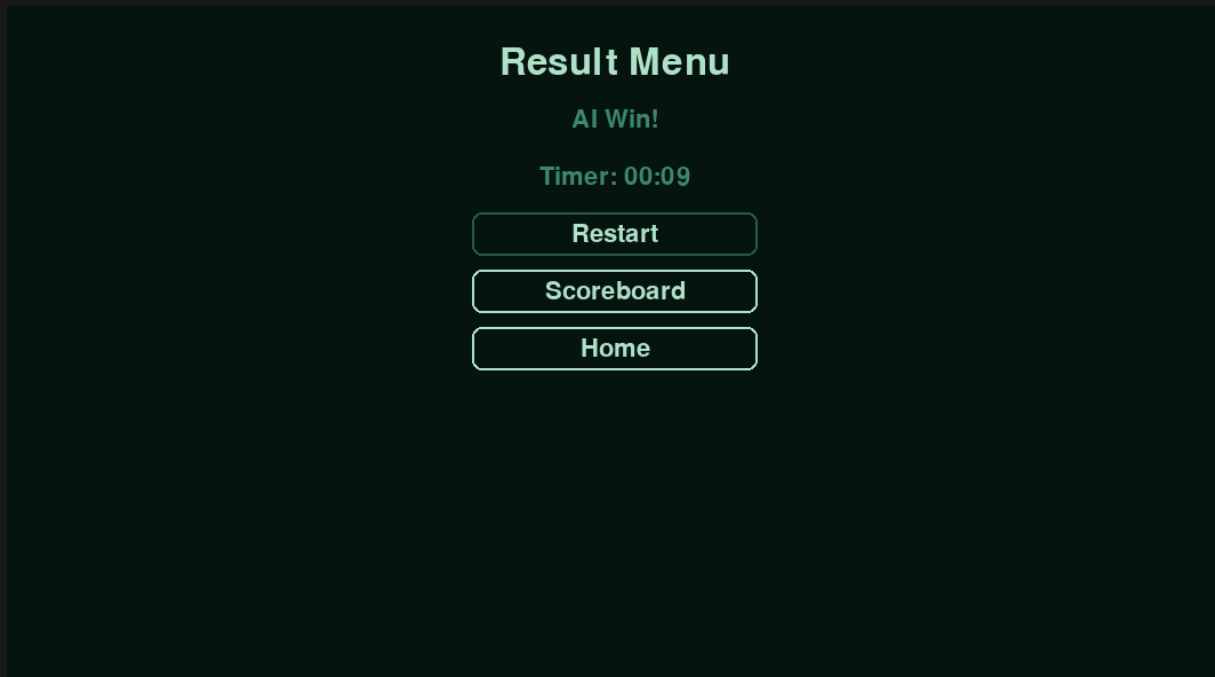
Pause Menu

Resume

Restart

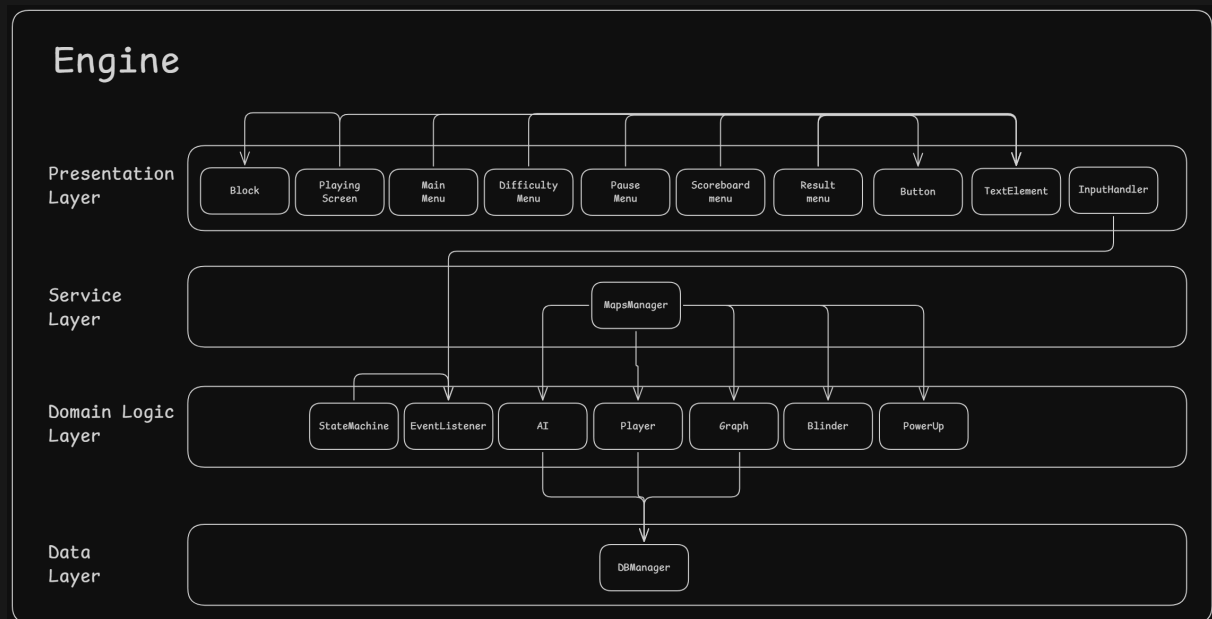
Home

Result Menu



Chapter 3

Development Plan (Architecture Diagram)



References

- pygame.org
- [geeksforgeeks.org](https://www.geeksforgeeks.org)
- [w3schools.com](https://www.w3schools.com)
- github.com
- neovim.io

Appendix

